



AI-Leveraged Backend Challenge: JVM Edition

1. Context: The Addi Sales Pipeline

In our company, we utilize a custom-made CRM to manage our sales pipeline where leads convert into prospects. To achieve our goals, we are automating the manual checks currently performed by sales agents.

Every lead in the CRM contains basic personal information: national identification number, birthdate, first name, last name, and email. Your mission is to build the automated orchestration layer that validates these leads before they are moved to the "Prospect" stage.

2. The Business Logic: Automated Lead Qualification

The criteria to turn a sales-qualified lead into a prospect involves passing three distinct validations:

- **National Registry Validation:** The person must exist in the external registry, and their data must match our local database.
- **Judicial Records Check:** The person must have no records in the national archives' external system.
- **Compliance Bureau (OFAC/Sanctions) & Caching:** A lightweight check against a compliance bureau. To optimize external calls and handle latency, implement a simple Persistent/Durable mechanism for these bureau responses. If the service is down, demonstrate resilience by gracefully handling the failure or triggering a manual review flow.
- **Prospect Qualification Score:** An internal system provides a random score between 0 and 100. A lead is converted only if the score is **greater than 60**.

Technical Execution Constraints

- **Parallelism:** The first two validations (Registry and Judicial) are non-dependent and must execute in parallel.
 - **Compliance Bureau:** Requires the successful output of the previous two validations to execute.
 - **Sequential Dependency:** The Qualification Score requires the successful clean output of the previous **Compliance Bureau** to execute.
-

3. The AI-Native Requirement

We expect you to use AI tools (e.g., Claude, Gemini, Cursor) to complete this challenge. We are evaluating your ability to orchestrate these tools to build high-quality, production-ready software with speed and reliability. We do not evaluate based on the specific tools or models you choose. If you already use paid tools feel free to leverage them. If you do not have paid access, you are strongly encouraged to use excellent free alternatives that support these agentic workflows. Options include `gemini-cli`, Gemini Assist, or Continue.dev. You can also use `qwen-code`, an open-source terminal agent offering a free tier via Qwen OAuth, `opencode`, an open-source coding agent with built-in access to free tier models, or Claude Code via OpenRouter's free tier.

4. Technical Considerations

- **Language:** Java (JVM).
- **Interface:** A simple **CLI** is sufficient; do not build a UI or client-server solution.
- **Spec-Driven Development:** Before implementing features, create a brief specification document that defines what you're building, key requirements, and design decisions. Your git history will show specs created before implementation.
- **Robust Process Documentation:** Include a comprehensive README that describes the entire process you took to solve the challenge.
- **AI Artifacts Required:** Submit all AI conversation history, complete prompt logs, and any context files used alongside your code submission.

- **Infrastructure:** Do not use external databases or message queues.
 - **External Systems:** Implement these as functions that respond with success or failure. You must **simulate latency** for these requests. You may use HTTP stubs or any technique of your choice.
-

5. Your repository must include:

A clear README file documenting your decisions, assumptions, and how you refined output to meet our standards. Include also your full project history (prompts/AI chat logs), what worked well and wrong using AI. Include all pending improvements to be done based in your assessment.